SCC Kernel Fortress: Executable Evaluation Gates for Runtime Compliance in Agentic Systems

Anonymous Author(s)
Affiliation
Address
email

Abstract

Stateful AI and agentic systems increasingly operate through tool-use loops, memory updates, external calls, policy constraints, recovery paths, and audit traces. These systems may emit logs, traces, policy declarations, or post-hoc explanations, but those artifacts do not by themselves verify that each runtime transition was admissible before it entered accepted execution history. We present SCC KERNEL FORTRESS, an executable evaluation framework for runtime-compliance claims in stateful agentic systems. The framework implements a deterministic pre-acceptance checker over a previous state, candidate next state, canonical event, audit event, proof-obligation bundle, and execution descriptor. A candidate transition is accepted only when its evidence satisfies the implemented Synthetic Cognitive Coding obligations: canonical encoding agreement, audit-chain consistency, protected-coordinate isolation, governance-state validity, risk-to-halt enforcement, halt absorption, lineage continuation, and normalized Exact/Stutter/SafeRefined step classification. The release includes a Rust source-of-truth checker, a TypeScript reference checker, committed golden vectors, binary canonical vectors, a negative corpus with named first-failure gates, property-test scaffolding, fuzz targets, a Lean/Coq formal bridge for the implemented v1 subset, federation evidence fixtures, numerical-stability bounds, a trusted computing base ledger, and CI release gates. The contribution is deliberately narrow: SCC KERNEL FORTRESS does not claim general AI safety, full theorem mechanization of the entire SCC canon, compiler correctness, operating-system correctness, or cryptographic soundness of SHA-256. Its claim is smaller and testable: it turns selected SCC obligations into reproducible executable gates that reject named transition failures before state acceptance. As an Evaluations & Datasets artifact, it contributes a reusable protocol, fixture corpus, and executable tool for testing transition-legality claims in stateful agentic runtimes. SCC KERNEL FORTRESS is therefore the executable boundary artifact; the Full SCC Canon is the formal authority it points back to.

1 Introduction

Modern AI systems are moving from stateless prediction toward stateful execution. They call tools, write files, update memory, route tasks, invoke policies, generate audit records, and sometimes attempt recovery after failure. In these settings, safety cannot be evaluated only by inspecting the final output. The critical question becomes whether the system's *runtime evolution* was lawful, auditable, and recoverable under the rules governing the runtime.

A stateful system can fail even when its visible output looks acceptable. It may erase lineage, mutate protected memory, lower a failure mode without authorization, bypass a halt condition, corrupt an audit chain, or accept a next state without machine-checkable evidence. These are not ordinary output-quality failures. They are transition-validity failures.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Most monitoring mechanisms are not designed to reject invalid transitions before acceptance. Logs can record what happened, but they do not necessarily prevent illegal state change. Prompts can declare policy, but they do not mechanically enforce legality. Schemas can validate object shape, but they do not verify audit continuity, protected-state preservation, governance bounds, risk escalation, halt absorption, or refinement correctness. Post-hoc audits can detect violations after the fact, but by then the invalid state may already have influenced later execution.

SCC KERNEL FORTRESS addresses this gap by treating runtime compliance as a pre-acceptance evaluation problem. Before a candidate next state can enter the accepted trajectory, it must pass a deterministic checker. If the proof-obligation bundle is absent, malformed, hash-inconsistent, audit-inconsistent, lineage-breaking, risk-invalid, halt-invalid, governance-invalid, or refinement-invalid, the transition is rejected.

This paper presents SCC KERNEL FORTRESS as the executable front layer of Synthetic Cognitive Coding, or SCC. The Full SCC Canon defines the broader constitutional-operational theory: typed state spaces, admissible trajectories, proof obligations, governance rules, risk and halt laws, memory and audit constraints, refinement semantics, implementation correspondence, distributed coherence, mechanization targets, and assumption boundaries [1]. The Engineering Handbook translates that canon into builder-facing discipline: implement the kernel first, keep it small, treat proof-obligation bundles as central artifacts, keep the checker deterministic, separate product logic from compliance logic, harden the trusted computing base, and reject ordinary execution when the bundle fails [2]. SCC KERNEL FORTRESS sits between them: it demonstrates how selected canonical obligations become deterministic runtime gates.

> **Artifact Claim.** Given a previous state, candidate next state, canonical event, audit event, proof-obligation bundle, and execution descriptor, SCC KERNEL FORTRESS deterministically accepts only transitions satisfying the implemented SCC obligations and rejects named violation classes before state acceptance.

> **Non-Claim.** SCC KERNEL FORTRESS does not prove general AI safety, full SCC theorem mechanization, compiler correctness, operating-system correctness, cryptographic soundness of SHA-256, or correctness of every possible SCC implementation.

Why this is an E&D artifact. The contribution is not a model-quality leaderboard or a new training recipe. It is a reusable evaluation protocol for a failure mode that ordinary output evaluation underspecifies: whether a stateful agent runtime can make a candidate transition legally admissible before the transition is committed. The artifact defines the evaluation unit, the accept/reject predicate, positive controls, negative controls, fixture-level first-failure expectations, and release gates. Other researchers can use the package in three ways: run the supplied checker against the committed corpus, implement an independent checker and compare first-failure behavior, or adapt the fixture protocol to audit their own stateful agent loops.

2 Operational terminology

The kernel preserves formal SCC vocabulary, but the evaluation artifact introduces each term through the operational problem it solves. Table 1 gives the translation used throughout this paper.

3 Evaluation protocol

SCC KERNEL FORTRESS is submitted as an evaluation artifact because its primary contribution is not a new model, dataset, or training method. Its contribution is a reproducible mechanism for evaluating whether runtime-compliance claims are enforced by deterministic acceptance logic. The evaluation question is simple: given a claimed compliant transition, does the runtime provide enough machine-checkable evidence for that transition to be accepted?

The evaluation unit is one candidate runtime transition. The checker receives six inputs and returns either acceptance or a named rejection. This is not a probability, policy score, or advisory warning; it is a deterministic acceptance predicate.

Table 1: SCC terminology as operational runtime concepts.

Internal term	Operational phrase
Proof-obligation bundle	Machine-checkable transition evidence
Constitutional admissibility	Runtime compliance rule satisfaction
Protected-coordinate isolation	Safety-critical state immutability
Halt absorption	Safety stop cannot be bypassed
Governance simplex	Valid bounded governance weights
Lineage continuation	Execution history continuity
SafeRefined	Risk-nonincreasing safe refinement

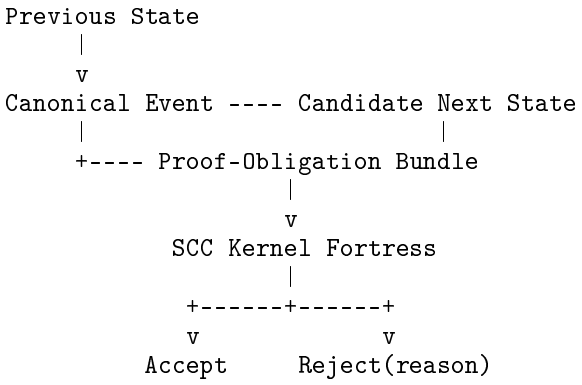

Figure 1: SCC KERNEL FORTRESS evaluates runtime compliance before state acceptance. A candidate transition enters the accepted trajectory only if its proof-obligation bundle, canonical hashes, audit event, risk/halt behavior, lineage, and refinement classification satisfy the implemented obligations.

4 Case study: runtime compliance failures in a stateful agent loop

A simple stateful agent loop can be represented as

$$State_t \to AgentEvent \to CandidateState_{t+1} \to POBundle \to Checker \to Accept/Reject. \tag{1}$$

SCC KERNEL FORTRESS evaluates whether the candidate transition is admissible before it becomes part of runtime history. Table 3 maps representative agent-loop compliance failures to committed negative-corpus fixtures. Each fixture is expected to reject at the named first-failure gate; a different first-failure gate is treated as a regression.

The point of the case study is not that SCC KERNEL FORTRESS solves every agent-safety problem. The point is that it catches a class of runtime-compliance failures that ordinary output evaluation can miss.

5 Baseline comparison

SCC KERNEL FORTRESS is best understood by contrast with mechanisms that record, advise, or partially validate behavior but do not necessarily reject invalid transitions before state acceptance. Table 4 is a qualitative capability comparison. It does not claim that all systems in a class lack stronger variants; it identifies which guarantees are provided by this artifact as packaged and tested.

The differentiator is pre-acceptance enforcement. SCC KERNEL FORTRESS does not merely record that a transition occurred; it determines whether the transition may be accepted.

Table 2: Evaluation protocol.

Protocol element	Definition
Evaluation unit	One candidate state transition.
Inputs	Previous state, candidate next state, canonical event, audit event, proof-obligation bundle, and execution descriptor.
Decision	Accept, or reject with a named reason.
Positive controls	Golden vectors for Exact, Stutter, and SafeRefined steps.
Negative controls	Named first-failure corpus for malformed or unlawful bundles.
Reproducibility	Rust/TypeScript checks, committed binary vectors, forbidden-import checks, and CI gates.

Table 3: Representative agent-loop failures mapped to committed negative-corpus fixtures.

Failure scenario	Runtime symptom	Committed fixture	First-failure gate
Missing evidence	Agent proposes a state update without a valid proof-obligation bundle.	`missing_required_field.json`	`missing_required_field`
Audit corruption	Runtime rewrites or mismatches the audit hash.	`audit_event_hash_bad.json`	`audit_event_hash_bad`
Risk escalation	Risk threshold is crossed but the system does not halt.	`risk_threshold_without_halt.json`	`risk_threshold_without_halt`
Halt escape	System exits halt without authorized recovery.	`halt_absorption_violation.json`	`halt_absorption_violation`
Governance corruption	Governance weights leave the valid simplex.	`invalid_simplex.json`	`invalid_simplex`
Lineage erasure	Runtime breaks or deletes continuation history.	`lineage_erasure.json`	`lineage_erasure`
Protected-state mutation	Ordinary execution mutates protected coordinates.	`protected_mutation.json`	`protected_mutation`
Refinement misclassification	Bundle declares a mode inconsistent with the checker classification.	`wrong_step_mode.json`	`wrong_step_mode`

6 Implemented obligations and checker pipeline

The checker is a predicate over a previous state s, next state s', event e, audit event a, proof-obligation bundle p, and execution descriptor Θ:

$$\mathrm{accepts}(p, s, s', e, a, \Theta) \in \{\mathsf{Ok}, \mathsf{Err}(r)\}. \tag{2}$$

It is pure: no wall-clock time, randomness, network calls, environment variables, mutable globals, telemetry, or product runtime decisions participate in the decision.

The Rust `accepts` function is the single gate. It rejects in this order: schema mismatch, checker-version mismatch, invalid mode, missing required PO field, canonical hash mismatch, domain violation, protected-coordinate mutation, halt violation, governance violation, risk violation, lineage violation, audit-chain break, and refinement violation. The order is intentional because negative corpus cases assert the first failing gate.

The v1 kernel defines typed records for `SCCState`, `CanonicalEvent`, `AuditEvent`, `POBundle`, and `ExecutionEnv`. All records are serialized by domain-separated binary encoders. Strings are UTF-8 with u32 lengths, integers are big-endian, maps are key-sorted, and floating point values are finite IEEE-754 binary64 in big-endian order with negative zero normalized. JSON is a carrier format only; the committed `.bin` vectors are the byte-level contract. The audit chain avoids circularity by hashing

Table 4: Pre-acceptance enforcement compared with common monitoring mechanisms.

Mechanism	Records	Pre-accept reject	Checks hash/audit	Enforces halt/risk	Protects state
Plain logs	Yes	No	No	No	No
Prompt policy	Weakly	No	No	No	No
JSON schema validation	Shape only	Partially	No	No	No
Post-hoc audit	Yes	No	Sometimes	No	No
Runtime policy wrapper	Sometimes	Sometimes	Rarely	Sometimes	Rarely
SCC KERNEL FORTRESS	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Table 5: Canon-to-executable mapping.

SCC source	Executable obligation	Negative/golden gate
Article F9, Article C11	Valid proof-obligation bundle or no lawful step.	`missing_required_field`
Article 29	Governance vector must remain in the simplex.	`invalid_simplex`
Article 30	Threshold risk must force halt.	`risk_threshold_without_halt`
Article 32	Halt is absorbing unless authorized recovery applies.	`halt_absorption_violation`
Article 35	Audit chain is deterministic and hash-bound.	`audit_event_hash_bad`
Article 36	Lineage must continue.	`lineage_erasure`
Articles 52–53	Step mode must be Exact, Stutter, or SafeRefined.	Three golden modes; `wrong_step_mode`
Articles 73, 80	Trusted computing base and external assumptions must be explicit.	`tcb_ledger.md`

the next state with the audit coordinate zeroed:

$$H_a = \mathrm{SHA256}(\mathrm{AuditEvent}(H_t, H(e), H(s_t), H_{body}(s_{t+1}), \Theta)). \tag{3}$$

The checker then requires both $p.\mathrm{audit\_event\_hash} = H_a$ and $s_{t+1}.\mathrm{audit\_hash} = H_a$.

7 Step classification, governance, risk, and halt

The canon separates full abstraction ρ from normalized simulation projection $\widehat{\rho}$ so that audit and lineage can advance during behavioral stuttering [1]. The implementation follows that split. The normalized projection drops administrative audit and lineage coordinates while retaining compute root, memory root, vector clock, governance weights, failure mode, risk vector, and protected root.

Given the induced exact successor $\widehat{T}(\widehat{\rho}(s), e)$, the classifier returns exactly one of:

$$\mathsf{Exact}: \quad \widehat{\rho}(s') = \widehat{T}(\widehat{\rho}(s), e), \tag{4}$$

$$\mathsf{Stutter}: \quad \widehat{\rho}(s') = \widehat{\rho}(s), \tag{5}$$

$$\mathsf{SafeRefined}: \quad \widehat{\rho}(s') \preceq_{safe} \widehat{T}(\widehat{\rho}(s), e). \tag{6}$$

The v1 safety preorder keeps structural coordinates stable, permits no greater total risk, and forbids lowering the failure mode. A bundle whose declared mode disagrees with the classifier is rejected.

Governance weights must lie in the probability simplex. The transition helper uses the standard sort-threshold-renormalize projection family for simplex or ℓ_1-ball constraints [4, 17]. The checker does not trust a projection witness blindly: it checks that the next vector lies in the simplex and that L2 drift is within the descriptor bound.

Risk is executable law in v1. If $\sum_i r_i \geq \Theta.\mathrm{risk\_halt\_threshold}$, then the next failure mode must be 3. If the previous failure mode is 3 and the bundle does not invoke an authorized recovery path, the next failure mode must remain 3. Ordinary execution also cannot silently lower severity.

8 Reproducibility and release gates

The anonymized supplementary package contains three accepted golden modes (Exact, Stutter, and SafeRefined), fourteen negative cases, JSON and binary vector forms, Rust tests, TypeScript tests,

property-test scaffolding, fuzz targets, schemas, a TCB ledger, and a CI workflow. The TypeScript reference suite contains 22 tests: three golden acceptance tests, fourteen negative rejection tests, one replay determinism test, and four stress tests covering memory-compression lineage, repaired-hash lineage rejection, simplex numerical stability, and federation quorum arithmetic. The release gate is:

Listing 1: Release gate used by local and CI validation.

```
python tools/validate_vectors.py
python tools/forbidden_imports.py
cd rust && cargo test --all-features
cd rust && cargo test --test negative_corpus
cd rust && cargo test --test golden_vectors
cd rust && cargo test --test proptest_invariants
cd rust && cargo test --test memory_lineage_compression
cd rust && cargo test --test numerical_stability
cd rust && cargo test --test federation_quorum
cd ts && npm test
python tools/validate_formal_artifacts.py
bash scripts/run_formal_gates.sh
```

The submitted artifact should be considered valid only when these commands, or the included CI workflow that executes the same checks on a Rust- and Node-equipped runner, complete successfully. The artifact also includes a local packaging transcript for vector validation, forbidden-import scanning, and the TypeScript reference suite. We do not use unobserved local commands as evidence; unsupported release claims must be backed by the supplied CI or by an independently reproduced local transcript.

Table 6: Evidence tiers for reproducing the artifact claim.

Tier	Reviewer action	Evidence produced
Static	Inspect README, schemas, TCB ledger, fixture manifests, and checker pipeline.	Confirms artifact scope, inputs, claims, and trust boundary.
Vector	Run `validate_vectors.py` and inspect committed `.bin` fixtures.	Confirms byte-level fixture consistency for golden and negative cases.
Reference	Run the TypeScript suite under Node 22+.	Confirms 22 deterministic reference checks over golden, negative, replay, compression-lineage, numerical, and federation arithmetic cases.
Authority	Run Rust release gates under a Rust-equipped runner or the included CI workflow.	Confirms source-of-truth checker behavior, negative-corpus first-failure gates, golden-vector acceptance, and property tests.

9 Artifact use cases

SCC KERNEL FORTRESS is packaged for use as an executable evaluation harness, not only as a reference implementation. Table 7 summarizes the intended reviewer and downstream use cases. Each use case is intentionally transition-level: the artifact evaluates a candidate state transition and its evidence, not the natural-language quality of a generated answer.

10 Mechanization, stress, federation, and numerical hardening

This release turns the prior mechanization roadmap into a concrete bridge for the implemented v1 subset. The supplement includes a Lean 4 micro-model of the gate, a seven-stage witness formalization, and a Coq proof fragment for required-bundle reflection and quorum-intersection arithmetic. The mechanized claim is intentionally scoped: in the small formal model, acceptance implies Prop-level v1 obligations such as required bundle fields, protected-coordinate preservation, halt absorption, risk-to-halt implication, lineage continuation, and stage-witness completeness. The bridge does not prove the entire SCC canon, compiler correctness, SHA-256 security, or operating-system correctness.

These additions strengthen the artifact claim without enlarging it. The single-node checker remains the core evaluation unit. Mechanization proves the boundary in a small formal model; Rust/Type-

Table 7: Executable artifact use cases.

Use case	What the evaluator does	Required evidence
Submission inspection	Run the release gate over the supplied package.	Golden modes accept; all negative fixtures reject at named first-failure gates.
Runtime regression test	Add the fixture corpus to a stateful agent runtime's CI.	No transition can commit unless the checker accepts its proof-obligation bundle.
Independent checker comparison	Reimplement the acceptance predicate in another language.	Decisions and first-failure gates match the committed manifests.
Policy-wrapper audit	Feed proposed state transitions through SCC KERNEL FORTRESS before logging or committing them.	Wrapper cannot substitute logs, prompts, or schemas for machine-checkable transition evidence.
Mechanization target	Translate the Rust gate and fixture obligations into proof-assistant lemmas.	The TCB ledger and canon mapping define what must be proven versus assumed.

Table 8: Hardening additions beyond the single-node fixture corpus.

Area	Added evidence	Reviewer handle
Mechanization	Lean Bool-to-Prop gate soundness, Lean seven-stage witness bridge, Coq required-bundle and quorum arithmetic proofs.	`formal/`, `mechanization_manifest.json`
Rust stress	Added memory-lineage compression, numerical-stability, and federation-quorum integration tests, plus a lineage fuzz target.	`rust/tests/*`, `rust/fuzz/*`
Memory/lineage	Compression stress repairs hashes first, then tests accepted lineage continuation and semantic lineage-erasure rejection.	`memory_lineage_compression.rs`
Federation	Distributed story is a certificate layer over single-node legality: quorum certificates, cross-shard lineage binding, and bounded halt propagation.	`federation/manifest.json`
Numerics	TCB ledger now specifies finite binary64 encoding, negative-zero normalization, simplex tolerance, drift bounds, and near-threshold risk discipline.	`docs/numerical_stability_tcb.md`
Navigation	One-page system map reduces doc sprawl and connects Canon, Handbook, Kernel Fortress, and Builder Scaffold.	`docs/SCC_SYSTEM_OVERVIEW_ONE_PAGER.md`

Script stress tests increase executable coverage; federation fixtures specify distributed evidence that composes with, rather than replaces, the single-node gate; numerical rules make floating-point trust explicit in the TCB ledger.

11 Threat model, limitations, and canon bridge

SCC KERNEL FORTRESS targets runtime-compliance failures in which a stateful system attempts to accept a transition that lacks required machine-checkable evidence, corrupts canonical hashes or audit continuity, mutates protected coordinates through ordinary execution, erases or severs lineage, crosses a risk threshold without entering halt, exits halt without authorized recovery, submits invalid governance-state updates, misdeclares its refinement mode, or depends on nondeterministic or unauthorized runtime inputs. This is an ML-evaluation contribution at the runtime layer: it evaluates transition legality in stateful agentic systems, rather than model likelihood, task score, or response preference.

It does not defend against every possible system compromise. It does not prove compiler correctness, operating-system correctness, hardware correctness, cryptographic primitive security, model-objective safety, or semantic harmlessness of generated content. It does not evaluate natural-language helpfulness, truthfulness, toxicity, bias, or preference alignment. The implemented v1 subset now includes a formal bridge and proof targets, but the artifact still does not claim full mechanical proof of the entire

SCC canon or of the surrounding compiler, operating system, cryptographic primitive, hardware, or deployment substrate. These limits are not footnotes; they are part of the artifact's trust boundary.

SCC KERNEL FORTRESS is not the whole SCC system. It is the executable gate. The Full SCC Canon provides the broader formal authority: state spaces, constitutional clauses, admissible domains, transition laws, proof-obligation requirements, governance and risk dynamics, audit determinism, lineage continuation, refinement semantics, implementation correspondence, distributed coherence, mechanization targets, and assumption boundaries [1]. The Engineering Handbook translates that authority into builder-facing instructions for a small deterministic kernel, proof-obligation bundles, TCB hardening, replay discipline, and release gates [2]. The executable artifact makes compliance testable; the canon supplies the constitutional-operational law behind the tests.

12 Related work

SCC KERNEL FORTRESS is adjacent to runtime verification, where monitors check properties of executions [7, 6], and to proof-carrying code, where consumers check explicit evidence before accepting code [11]. Reference-monitor and security-policy work motivate a mediating enforcement point for security-relevant decisions [3, 16]. Policy-as-code systems such as Open Policy Agent/Rego decouple policy decisions from application code and express policies over structured data [12]. Secure audit-log work studies tamper-evident logging after events occur [15]. SCC KERNEL FORTRESS is complementary: it makes the transition itself carry typed evidence, canonical byte commitments, risk/halt obligations, lineage constraints, and named rejection gates before acceptance.

Agent and tool-use evaluations such as AgentBench, ToolLLM/ToolBench, ToolSandbox, and ToolEmu evaluate interactive task performance, tool-use capability, or risk discovery in tool-rich environments [8, 13, 9, 14]. ML safety evaluations and red-teaming frameworks study harmful outputs, refusal robustness, and adversarial prompt discovery [5, 10]. SCC KERNEL FORTRESS occupies a different evaluation layer: it does not score the semantic quality of an agent's response, but checks whether the runtime transition that produced or followed that response was admissible before state acceptance.

13 Conclusion

SCC KERNEL FORTRESS moves SCC enforcement from advisory documentation into an executable proof-of-boundary. It evaluates one candidate state transition at a time, requires machine-checkable evidence, validates canonical hashes and audit continuity, enforces protected-state, governance, risk, halt, lineage, and refinement obligations, and rejects named failure classes before acceptance.

The bridge to the Full SCC Canon is the central contribution. SCC KERNEL FORTRESS shows that selected canonical obligations can become machinery. The canon explains the law behind that machinery. Together, they make runtime compliance reviewable, reproducible, and falsifiable: A system that cannot prove the legality of its next step has no right to take it.

References

[1] Anonymous. *Synthetic Cognitive Coding (SCC): Formal Constitutional-Operational Specification, Final Canon, Scholarly Edition*. Anonymized supplemental manuscript, 2026.

[2] Anonymous. *Synthetic Cognitive Coding (SCC): Engineering Handbook, Hardened Builder Edition*. Anonymized supplemental manuscript, 2026.

[3] James P. Anderson. *Computer Security Technology Planning Study*. Technical Report ESD-TR-73-51, 1972.

[4] John C. Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections onto the ℓ_1-ball for learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning*, pages 272–279, 2008.

[5] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, Andy Jones, Sam Bowman, Anna

Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Nelson Elhage, Sheer El-Showk, Stanislav Fort, Zac Hatfield-Dodds, Tom Henighan, Danny Hernandez, Tristan Hume, Josh Jacobson, Scott Johnston, Shauna Kravec, Catherine Olsson, Sam Ringer, Eli Tran-Johnson, Dario Amodei, Tom Brown, Nicholas Joseph, Sam McCandlish, Chris Olah, Jared Kaplan, and Jack Clark. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. arXiv:2209.07858, 2022.

[6] Klaus Havelund and Grigore Rosu. Efficient monitoring of safety properties. *International Journal on Software Tools for Technology Transfer*, 6(2):158–173, 2004.

[7] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.

[8] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

[9] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. arXiv:2408.04682, 2025.

[10] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[11] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119, 1997.

[12] Open Policy Agent Authors. Open Policy Agent documentation: Policy language and Rego. 2026. URL: `https://www.openpolicyagent.org/docs/policy-language`.

[13] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024.

[14] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. arXiv:2309.15817, 2023.

[15] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999.

[16] Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.

[17] Weiran Wang and Miguel A. Carreira-Perpinan. Projection onto the probability simplex: An efficient algorithm with a simple proof, and an application. arXiv:1309.1541, 2013.

A Supplementary reproducibility notes

The release package is intended to be evaluated as an executable artifact. The anonymized archive should contain the Rust crate, TypeScript reference checker, schemas, golden vectors, binary canonical vectors, negative corpus, TCB ledger, validation tools, Makefile, and CI workflow. The negative-corpus manifest contains fourteen cases: `audit_event_hash_bad`, `checker_version_mismatch`, `descriptor_mismatch_audit`, `event_hash_bad`, `halt_absorption_violation`, `invalid_simplex`, `lineage_erasure`, `missing_required_field`, `next_hash_bad`, `prev_hash_bad`, `protected_mutation`, `risk_threshold_without_halt`, `schema_mismatch`, and `wrong_step_mode`. Each case is committed as a JSON fixture and checked against its expected first-failure gate.

B Supplementary mechanization and distributed notes

The supplement also contains `formal/lean`, `formal/coq`, `mechanization_manifest.json`, `federation/`, `docs/numerical_stability_tcb.md`, and `docs/SCC_SYSTEM_OVERVIEW_ONE_PAGER.md`. The formal files are part of the evidence package; proof-assistant execution must be reported from a proof-equipped runner and must not be inferred from the static source tree alone.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the artifact claim and non-claim, and Sections 2–11 delimit the operational vocabulary, evaluation protocol, implemented obligations, artifact use cases, mechanization/federation/numerical hardening, threat model, evidence package, and limitations.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 11 states the threat model and limits, including non-claims about general AI safety, compiler/OS/hardware correctness, cryptographic primitive security, full canon mechanization, and proof of deployment substrates.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.

- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper includes formal notation for the acceptance predicate and step classification, but it does not introduce new theorems or proofs. Formal authority is cited to the anonymized SCC canon.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 7 gives release-gate commands, fixture manifests, vector validation, negative corpus requirements, evidence tiers, formal bridge validation, and CI workflow requirements needed to reproduce the artifact-level claims.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.

- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The submission is intended to include an anonymized supplementary archive containing the Rust checker, TypeScript checker, schemas, golden vectors, negative corpus, formal bridge files, federation fixtures, numerical-stability ledger, validation tools, TCB ledger, Makefile, Dockerfile, run script, and CI workflow.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Sections 3–10 specify the evaluation unit, inputs, deterministic decision rule, positive controls, negative controls, artifact use cases, mechanization/federation/numerical hardening, evidence tiers, and commands used for artifact validation. There are no model-training experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [N/A]

Justification: The artifact evaluation is deterministic and consists of exact accept/reject checks over committed fixtures; no stochastic experiment or statistical estimator is used.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The reproducibility section specifies a CPU-only release gate using Python 3, Rust 1.78+ or compatible stable, Node 22+, and optional Lean/Coq for formal-gate execution, with no GPU training resources; the supplement also provides Docker/CI instructions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.

- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work introduces a safety/compliance evaluation artifact, does not use human-subject data, and includes explicit threat-model, misuse, and limitation boundaries.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Sections 1, 8, 9, and the appendix discuss the intended positive impact of pre-acceptance compliance checking and the negative risks of over-claiming, misuse, or treating the artifact as general AI safety.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

Justification: The paper does not release a pretrained model, image generator, scraped dataset, or other high-misuse data/model asset. The release is an executable checker and test fixtures.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Prior work and external assets are cited in the references. The supplementary artifact should include its license, TCB ledger, dependency assumptions, and release manifest.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new asset is the executable evaluation artifact. It is documented in the paper and supplementary archive with schemas, fixtures, validation commands, release gates, and limitation notes.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.

- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The research does not involve crowdsourcing, user studies, human-subject experiments, or participant compensation.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The research does not involve human subjects or participants, so IRB or equivalent approval is not applicable.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: No LLM or agent is an original or non-standard component of the core method. The artifact is a deterministic checker over committed runtime-transition records.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.